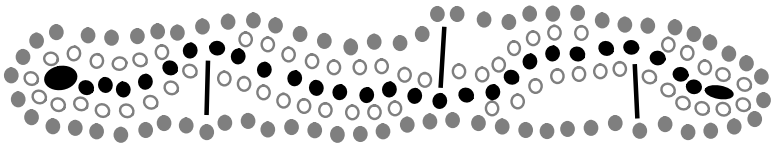
Chapter 7 Linked Lists



Contents

7.1 Singly Linked Lists 256
7.1.1 Implementing a Stack with a Singly Linked List 261
7.1.2 Implementing a Queue with a Singly Linked List 264
7.2 Circularly Linked Lists 266
7.2.1 Round-Robin Schedulers 267
7.2.2 Implementing a Queue with a Circularly Linked List 268
7.3 Doubly Linked Lists 270
7.3.1 Basic Implementation of a Doubly Linked List 273
7.3.2 Implementing a Deque with a Doubly Linked List 275
7.4 The Positional List ADT 277
7.4.1 The Positional List Abstract Data Type 279
7.4.2 Doubly Linked List Implementation 281
7.5 Sorting a Positional List 285
7.6 Case Study: Maintaining Access Frequencies 286
7.6.1 Using a Sorted List 286
7.6.2 Using a List with the Move-to-Front Heuristic 289
7.7 Link-Based vs. Array-Based Sequences 292
7.8 Exercises 294

In Chapter 5 we carefully examined Python's array-based list class, and in Chapter 6 we demonstrated use of that class in implementing the classic stack, queue, and dequeue ADTs. Python's list class is highly optimized, and often a great choice for storage. With that said, there are some notable disadvantages:

1. The length of a dynamic array might be longer than the actual number of elements that it stores.
2. Amortized bounds for operations may be unacceptable in real-time systems.
3. Insertions and deletions at interior positions of an array are expensive.

In this chapter, we introduce a data structure known as a ***linked list***, which provides an alternative to an array-based sequence (such as a Python list). Both array-based sequences and linked lists keep elements in a certain order, but using a very different style. An array provides the more centralized representation, with one large chunk of memory capable of accommodating references to many elements. A linked list, in contrast, relies on a more distributed representation in which a lightweight object, known as a ***node***, is allocated for each element. Each node maintains a reference to its element and one or more references to neighboring nodes in order to collectively represent the linear order of the sequence.

We will demonstrate a trade-off of advantages and disadvantages when contrasting array-based sequences and linked lists. Elements of a linked list cannot be efficiently accessed by a numeric index k, and we cannot tell just by examining a node if it is the second, fifth, or twentieth node in the list. However, linked lists avoid the three disadvantages noted above for array-based sequences.

7.1 Singly Linked Lists

A ***singly linked list***, in its simplest form, is a collection of ***nodes*** that collectively form a linear sequence. Each node stores a reference to an object that is an element of the sequence, as well as a reference to the next node of the list (see Figures 7.1 and 7.2).

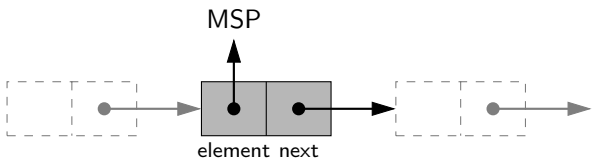


Figure 7.1: Example of a node instance that forms part of a singly linked list. The node's element member references an arbitrary object that is an element of the sequence (the airport code MSP, in this example), while the next member references the subsequent node of the linked list (or None if there is no further node).

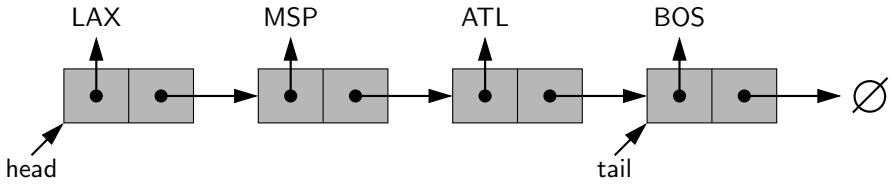


Figure 7.2: Example of a singly linked list whose elements are strings indicating airport codes. The list instance maintains a member named head that identifies the first node of the list, and in some applications another member named tail that identifies the last node of the list. The **None** object is denoted as Ø.

The first and last node of a linked list are known as the ***head*** and ***tail*** of the list, respectively. By starting at the head, and moving from one node to another by following each node's next reference, we can reach the tail of the list. We can identify the tail as the node having None as its next reference. This process is commonly known as ***traversing*** the linked list. Because the next reference of a node can be viewed as a ***link*** or ***pointer*** to another node, the process of traversing a list is also known as ***link hopping*** or ***pointer hopping***.

A linked list's representation in memory relies on the collaboration of many objects. Each node is represented as a unique object, with that instance storing a reference to its element and a reference to the next node (or None). Another object represents the linked list as a whole. Minimally, the linked list instance must keep a reference to the head of the list. Without an explicit reference to the head, there would be no way to locate that node (or indirectly, any others). There is not an absolute need to store a direct reference to the tail of the list, as it could otherwise be located by starting at the head and traversing the rest of the list. However, storing an explicit reference to the tail node is a common convenience to avoid such a traversal. In similar regard, it is common for the linked list instance to keep a count of the total number of nodes that comprise the list (commonly described as the ***size*** of the list), to avoid the need to traverse the list to count the nodes.

For the remainder of this chapter, we continue to illustrate nodes as objects, and each node's "next" reference as a pointer. However, for the sake of simplicity, we illustrate a node's element embedded directly within the node structure, even though the element is, in fact, an independent object. For example, Figure 7.3 is a more compact illustration of the linked list from Figure 7.2.

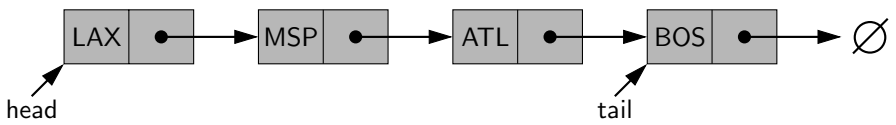


Figure 7.3: A compact illustration of a singly linked list, with elements embedded in the nodes (rather than more accurately drawn as references to external objects).

Inserting an Element at the Head of a Singly Linked List

An important property of a linked list is that it does not have a predetermined fixed size; it uses space proportionally to its current number of elements. When using a singly linked list, we can easily insert an element at the head of the list, as shown in Figure 7.4, and described with pseudo-code in Code Fragment 7.1. The main idea is that we create a new node, set its element to the new element, set its next link to refer to the current head, and then set the list's head to point to the new node.

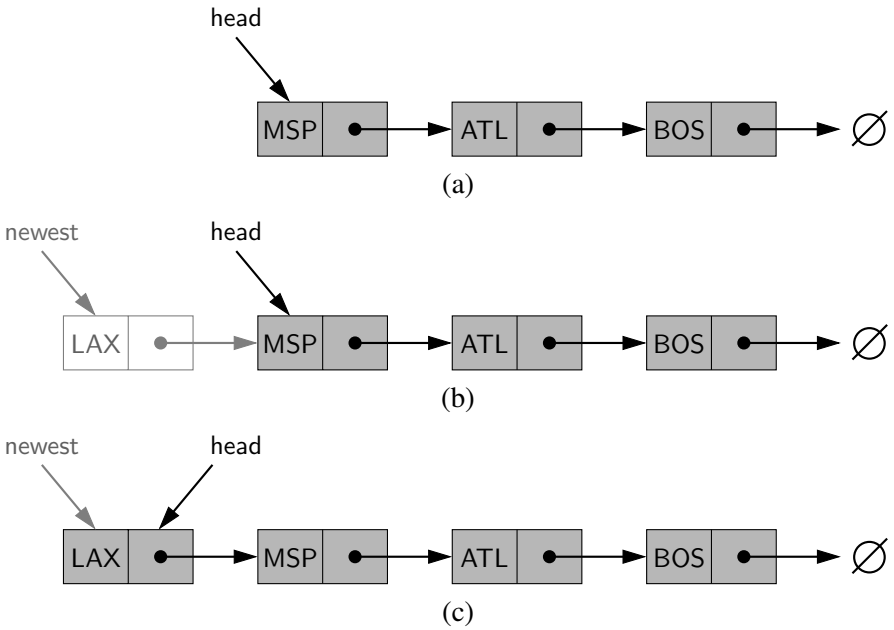


Figure 7.4: Insertion of an element at the head of a singly linked list: (a) before the insertion; (b) after creation of a new node; (c) after reassignment of the head reference.

Algorithm add_first(L, e):

```
newest = Node(e)      {create new node instance storing reference to element e}
newest.next = L.head  {set new node's next to reference the old head node}
L.head = newest       {set variable head to reference the new node}
L.size = L.size + 1   {increment the node count}
```

Code Fragment 7.1: Inserting a new element at the beginning of a singly linked list L. Note that we set the next pointer of the new node ***before*** we reassign variable L.head to it. If the list were initially empty (i.e., L.head is None), then a natural consequence is that the new node has its next reference set to None.

Inserting an Element at the Tail of a Singly Linked List

We can also easily insert an element at the tail of the list, provided we keep a reference to the tail node, as shown in Figure 7.5. In this case, we create a new node, assign its next reference to None, set the next reference of the tail to point to this new node, and then update the tail reference itself to this new node. We give the details in Code Fragment 7.2.

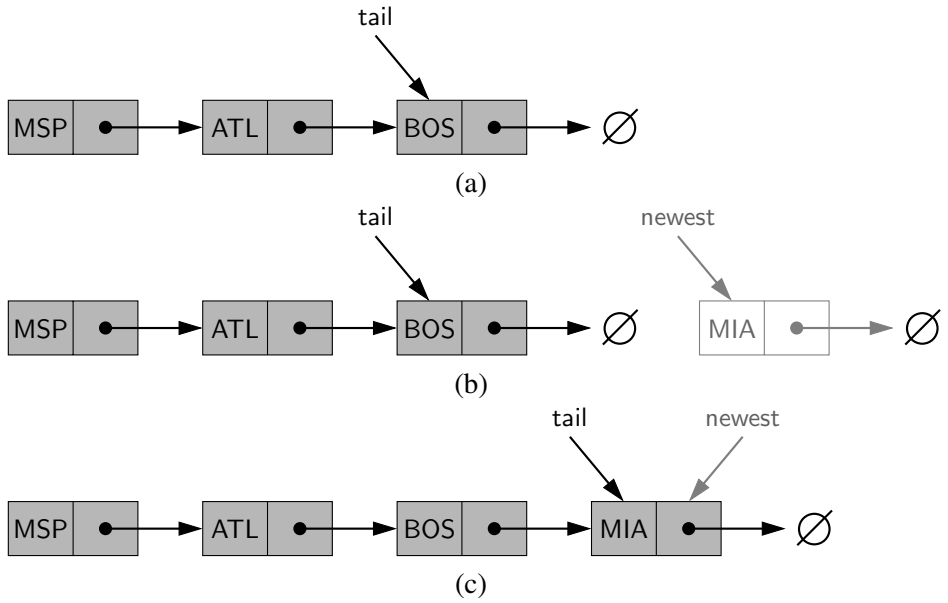


Figure 7.5: Insertion at the tail of a singly linked list: (a) before the insertion; (b) after creation of a new node; (c) after reassignment of the tail reference. Note that we must set the next link of the tail in (b) before we assign the tail variable to point to the new node in (c).

Algorithm add_last(L, e):

```
newest = Node(e)      {create new node instance storing reference to element e}
newest.next = None    {set new node's next to reference the None object}
L.tail.next = newest  {make old tail node point to new node}
L.tail = newest       {set variable tail to reference the new node}
L.size = L.size + 1   {increment the node count}
```

Code Fragment 7.2: Inserting a new node at the end of a singly linked list. Note that we set the next pointer for the old tail node ***before*** we make variable tail point to the new node. This code would need to be adjusted for inserting onto an empty list, since there would not be an existing tail node.

Removing an Element from a Singly Linked List

Removing an element from the ***head*** of a singly linked list is essentially the reverse operation of inserting a new element at the head. This operation is illustrated in Figure 7.6 and given in detail in Code Fragment 7.3.

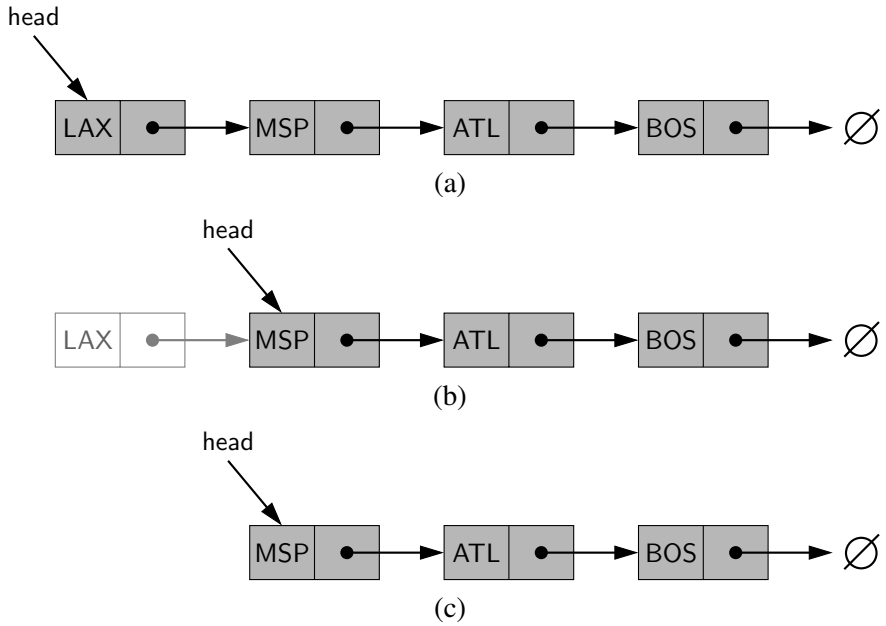


Figure 7.6: Removal of an element at the head of a singly linked list: (a) before the removal; (b) after "linking out" the old head; (c) final configuration.

Algorithm remove_first(L):
  if L.head is None **then**
    Indicate an error: the list is empty.
  L.head = L.head.next {make head point to next node (or None)}
  L.size = L.size − 1 {decrement the node count}

Code Fragment 7.3: Removing the node at the beginning of a singly linked list.

Unfortunately, we cannot easily delete the last node of a singly linked list. Even if we maintain a tail reference directly to the last node of the list, we must be able to access the node ***before*** the last node in order to remove the last node. But we cannot reach the node before the tail by following next links from the tail. The only way to access this node is to start from the head of the list and search all the way through the list. But such a sequence of link-hopping operations could take a long time. If we want to support such an operation efficiently, we will need to make our list ***doubly linked*** (as we do in Section 7.3).

7.1.1 Implementing a Stack with a Singly Linked List

In this section, we demonstrate use of a singly linked list by providing a complete Python implementation of the stack ADT (see Section 6.1). In designing such an implementation, we need to decide whether to model the top of the stack at the head or at the tail of the list. There is clearly a best choice here; we can efficiently insert and delete elements in constant time only at the head. Since all stack operations affect the top, we orient the top of the stack at the head of our list.

To represent individual nodes of the list, we develop a lightweight _Node class. This class will never be directly exposed to the user of our stack class, so we will formally define it as a nonpublic, nested class of our eventual LinkedStack class (see Section 2.5.1 for discussion of nested classes). The definition of the _Node class is shown in Code Fragment 7.4.

```
class _Node:
  """Lightweight, nonpublic class for storing a singly linked node."""
  __slots__ = '_element', '_next'          # streamline memory usage

  def __init__(self, element, next):       # initialize node's fields
    self._element = element                # reference to user's element
    self._next = next                      # reference to next node
```

Code Fragment 7.4: A lightweight _Node class for a singly linked list.

A node has only two instance variables: _element and _next. We intentionally define __slots__ to streamline the memory usage (see page 99 of Section 2.5.1 for discussion), because there may potentially be many node instances in a single list. The constructor of the _Node class is designed for our convenience, allowing us to specify initial values for both fields of a newly created node.

A complete implementation of our LinkedStack class is given in Code Fragments 7.5 and 7.6. Each stack instance maintains two variables. The _head member is a reference to the node at the head of the list (or None, if the stack is empty). We keep track of the current number of elements with the _size instance variable, for otherwise we would be forced to traverse the entire list to count the number of elements when reporting the size of the stack.

The implementation of push essentially mirrors the pseudo-code for insertion at the head of a singly linked list as outlined in Code Fragment 7.1. When we push a new element e onto the stack, we accomplish the necessary changes to the linked structure by invoking the constructor of the _Node class as follows:

```
self._head = self._Node(e, self._head)      # create and link a new node
```

Note that the _next field of the new node is set to the ***existing*** top node, and then self._head is reassigned to the new node.

```
1  class LinkedStack:
2    """LIFO Stack implementation using a singly linked list for storage."""
3
4    #---------------------------- nested _Node class ----------------------------
5    class _Node:
6      """Lightweight, nonpublic class for storing a singly linked node."""
7      __slots__ = '_element', '_next'         # streamline memory usage
8
9      def __init__(self, element, next):      # initialize node's fields
10       self._element = element               # reference to user's element
11       self._next = next                     # reference to next node
12
13   #------------------------------- stack methods -------------------------------
14   def __init__(self):
15     """Create an empty stack."""
16     self._head = None                       # reference to the head node
17     self._size = 0                          # number of stack elements
18
19   def __len__(self):
20     """Return the number of elements in the stack."""
21     return self._size
22
23   def is_empty(self):
24     """Return True if the stack is empty."""
25     return self._size == 0
26
27   def push(self, e):
28     """Add element e to the top of the stack."""
29     self._head = self._Node(e, self._head)    # create and link a new node
30     self._size += 1
31
32   def top(self):
33     """Return (but do not remove) the element at the top of the stack.
34
35     Raise Empty exception if the stack is empty.
36     """
37     if self.is_empty():
38       raise Empty('Stack is empty')
39     return self._head._element              # top of stack is at head of list
```

Code Fragment 7.5: Implementation of a stack ADT using a singly linked list (continued in Code Fragment 7.6).

```
40    def pop(self):
41      """Remove and return the element from the top of the stack (i.e., LIFO).
42
43      Raise Empty exception if the stack is empty.
44      """
45      if self.is_empty( ):
46        raise Empty('Stack is empty')
47      answer = self._head._element
48      self._head = self._head._next              # bypass the former top node
49      self._size -= 1
50      return answer
```

Code Fragment 7.6: Implementation of a stack ADT using a singly linked list (continued from Code Fragment 7.5).

When implementing the top method, the goal is to return the ***element*** that is at the top of the stack. When the stack is empty, we raise an Empty exception, as originally defined in Code Fragment 6.1 of Chapter 6. When the stack is nonempty, self._head is a reference to the first ***node*** of the linked list. The top element can be identified as self._head._element.

Our implementation of pop essentially mirrors the pseudo-code given in Code Fragment 7.3, except that we maintain a local reference to the element that is stored at the node that is being removed, and we return that element to the caller of pop.

The analysis of our LinkedStack operations is given in Table 7.1. We see that all of the methods complete in ***worst-case*** constant time. This is in contrast to the amortized bounds for the ArrayStack that were given in Table 6.2.

Operation	Running Time
S.push(e)	$O(1)$
S.pop( )	$O(1)$
S.top( )	$O(1)$
len(S)	$O(1)$
S.is_empty( )	$O(1)$

Table 7.1: Performance of our LinkedStack implementation. All bounds are worst-case and our space usage is $O(n)$, where n is the current number of elements in the stack.

7.1.2 Implementing a Queue with a Singly Linked List

As we did for the stack ADT, we can use a singly linked list to implement the queue ADT while supporting worst-case $O(1)$-time for all operations. Because we need to perform operations on both ends of the queue, we will explicitly maintain both a _head reference and a _tail reference as instance variables for each queue. The natural orientation for a queue is to align the front of the queue with the head of the list, and the back of the queue with the tail of the list, because we must be able to enqueue elements at the back, and dequeue them from the front. (Recall from the introduction of Section 7.1 that we are unable to efficiently remove elements from the tail of a singly linked list.) Our implementation of a LinkedQueue class is given in Code Fragments 7.7 and 7.8.

```
1  class LinkedQueue:
2    """FIFO queue implementation using a singly linked list for storage."""
3
4    class _Node:
5      """Lightweight, nonpublic class for storing a singly linked node."""
6      (omitted here; identical to that of LinkedStack._Node)
7
8    def __init__(self):
9      """Create an empty queue."""
10     self._head = None
11     self._tail = None
12     self._size = 0                          # number of queue elements
13
14   def __len__(self):
15     """Return the number of elements in the queue."""
16     return self._size
17
18   def is_empty(self):
19     """Return True if the queue is empty."""
20     return self._size == 0
21
22   def first(self):
23     """Return (but do not remove) the element at the front of the queue."""
24     if self.is_empty( ):
25       raise Empty('Queue is empty')
26     return self._head._element              # front aligned with head of list
```

Code Fragment 7.7: Implementation of a queue ADT using a singly linked list (continued in Code Fragment 7.8).

```
27    def dequeue(self):
28      """Remove and return the first element of the queue (i.e., FIFO).
29
30      Raise Empty exception if the queue is empty.
31      """
32      if self.is_empty( ):
33        raise Empty('Queue is empty')
34      answer = self._head._element
35      self._head = self._head._next
36      self._size -= 1
37      if self.is_empty( ):                    # special case as queue is empty
38        self._tail = None                     # removed head had been the tail
39      return answer
40
41    def enqueue(self, e):
42      """Add an element to the back of queue."""
43      newest = self._Node(e, None)            # node will be new tail node
44      if self.is_empty( ):
45        self._head = newest                   # special case: previously empty
46      else:
47        self._tail._next = newest
48      self._tail = newest                     # update reference to tail node
49      self._size += 1
```

Code Fragment 7.8: Implementation of a queue ADT using a singly linked list (continued from Code Fragment 7.7).

Many aspects of our implementation are similar to that of the LinkedStack class, such as the definition of the nested _Node class. Our implementation of dequeue for LinkedQueue is similar to that of pop for LinkedStack, as both remove the head of the linked list. However, there is a subtle difference because our queue must accurately maintain the _tail reference (no such variable was maintained for our stack). In general, an operation at the head has no effect on the tail, but when dequeue is invoked on a queue with one element, we are simultaneously removing the tail of the list. We therefore set self._tail to None for consistency.

There is a similar complication in our implementation of enqueue. The newest node always becomes the new tail. Yet a distinction is made depending on whether that new node is the only node in the list. In that case, it also becomes the new head; otherwise the new node must be linked immediately after the existing tail node.

In terms of performance, the LinkedQueue is similar to the LinkedStack in that all operations run in worst-case constant time, and the space usage is linear in the current number of elements.

7.2 Circularly Linked Lists

In Section 6.2.2, we introduced the notion of a "circular" array and demonstrated its use in implementing the queue ADT. In reality, the notion of a circular array was artificial, in that there was nothing about the representation of the array itself that was circular in structure. It was our use of modular arithmetic when "advancing" an index from the last slot to the first slot that provided such an abstraction.

In the case of linked lists, there is a more tangible notion of a circularly linked list, as we can have the tail of the list use its next reference to point back to the head of the list, as shown in Figure 7.7. We call such a structure a ***circularly linked list***.

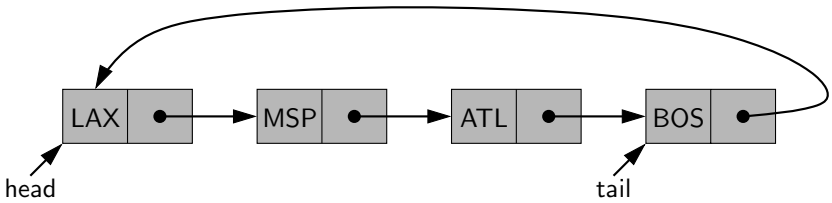


Figure 7.7: Example of a singly linked list with circular structure.

A circularly linked list provides a more general model than a standard linked list for data sets that are cyclic, that is, which do not have any particular notion of a beginning and end. Figure 7.8 provides a more symmetric illustration of the same circular list structure as Figure 7.7.

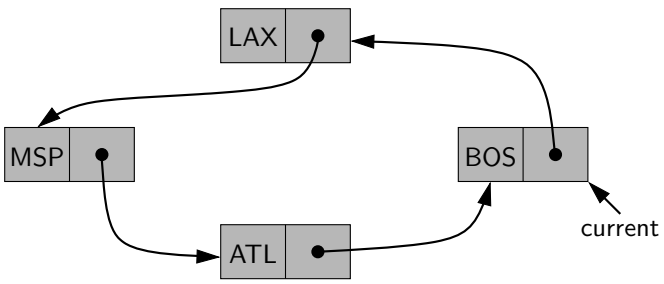


Figure 7.8: Example of a circular linked list, with current denoting a reference to a select node.

A circular view similar to Figure 7.8 could be used, for example, to describe the order of train stops in the Chicago loop, or the order in which players take turns during a game. Even though a circularly linked list has no beginning or end, per se, we must maintain a reference to a particular node in order to make use of the list. We use the identifier ***current*** to describe such a designated node. By setting current = current.next, we can effectively advance through the nodes of the list.

7.2.1 Round-Robin Schedulers

To motivate the use of a circularly linked list, we consider a ***round-robin*** scheduler, which iterates through a collection of elements in a circular fashion and "services" each element by performing a given action on it. Such a scheduler is used, for example, to fairly allocate a resource that must be shared by a collection of clients. For instance, round-robin scheduling is often used to allocate slices of CPU time to various applications running concurrently on a computer.

A round-robin scheduler could be implemented with the general queue ADT, by repeatedly performing the following steps on queue Q (see Figure 7.9):

1. $e = Q.\mathsf{dequeue}()$
2. Service element e
3. $Q.\mathsf{enqueue}(e)$

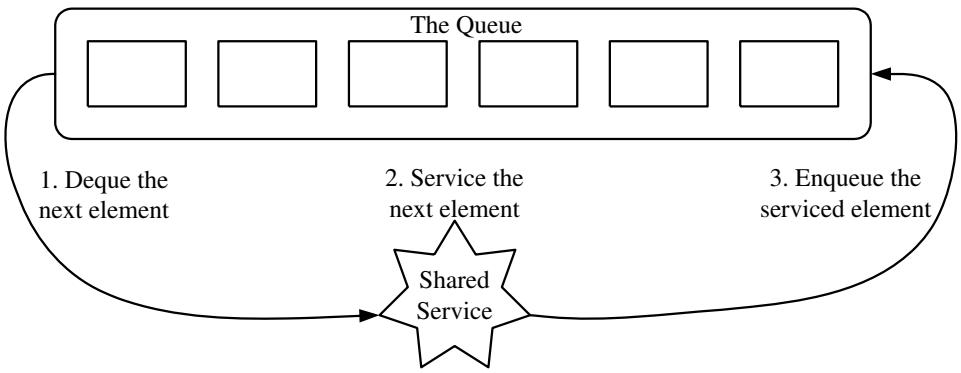


Figure 7.9: The three iterative steps for round-robin scheduling using a queue.

If we use of the LinkedQueue class of Section 7.1.2 for such an application, there is unnecessary effort in the combination of a dequeue operation followed soon after by an enqueue of the same element. One node is removed from the list, with appropriate adjustments to the head of the list and the size decremented, and then a new node is created to reinsert at the tail of the list and the size is incremented.

If using a circularly linked list, the effective transfer of an item from the "head" of the list to the "tail" of the list can be accomplished by advancing a reference that marks the boundary of the queue. We will next provide an implementation of a CircularQueue class that supports the entire queue ADT, together with an additional method, rotate( ), that moves the first element of the queue to the back. (A similar method is supported by the deque class of Python's collections module; see Table 6.4.) With this operation, a round-robin schedule can more efficiently be implemented by repeatedly performing the following steps:

1. Service element $Q.\mathsf{front}()$
2. $Q.\mathsf{rotate}()$

7.2.2 Implementing a Queue with a Circularly Linked List

To implement the queue ADT using a circularly linked list, we rely on the intuition of Figure 7.7, in which the queue has a head and a tail, but with the next reference of the tail linked to the head. Given such a model, there is no need for us to explicitly store references to both the head and the tail; as long as we keep a reference to the tail, we can always find the head by following the tail's next reference.

Code Fragments 7.9 and 7.10 provide an implementation of a CircularQueue class based on this model. The only two instance variables are _tail, which is a reference to the tail node (or None when empty), and _size, which is the current number of elements in the queue. When an operation involves the front of the queue, we recognize self._tail._next as the head of the queue. When enqueue is called, a new node is placed just after the tail but before the current head, and then the new node becomes the tail.

In addition to the traditional queue operations, the CircularQueue class supports a rotate method that more efficiently enacts the combination of removing the front element and reinserting it at the back of the queue. With the circular representation, we simply set self._tail = self._tail._next to make the old head become the new tail (with the node after the old head becoming the new head).

```
1  class CircularQueue:
2    """Queue implementation using circularly linked list for storage."""
3
4    class _Node:
5      """Lightweight, nonpublic class for storing a singly linked node."""
6      (omitted here; identical to that of LinkedStack._Node)
7
8    def __init__(self):
9      """Create an empty queue."""
10     self._tail = None                    # will represent tail of queue
11     self._size = 0                       # number of queue elements
12
13   def __len__(self):
14     """Return the number of elements in the queue."""
15     return self._size
16
17   def is_empty(self):
18     """Return True if the queue is empty."""
19     return self._size == 0
```

Code Fragment 7.9: Implementation of a CircularQueue class, using a circularly linked list as storage (continued in Code Fragment 7.10).

```
20    def first(self):
21      """Return (but do not remove) the element at the front of the queue.
22
23      Raise Empty exception if the queue is empty.
24      """
25      if self.is_empty( ):
26        raise Empty('Queue is empty')
27      head = self._tail._next
28      return head._element
29
30    def dequeue(self):
31      """Remove and return the first element of the queue (i.e., FIFO).
32
33      Raise Empty exception if the queue is empty.
34      """
35      if self.is_empty( ):
36        raise Empty('Queue is empty')
37      oldhead = self._tail._next
38      if self._size == 1:                    # removing only element
39        self._tail = None                    # queue becomes empty
40      else:
41        self._tail._next = oldhead._next     # bypass the old head
42      self._size -= 1
43      return oldhead._element
44
45    def enqueue(self, e):
46      """Add an element to the back of queue."""
47      newest = self._Node(e, None)           # node will be new tail node
48      if self.is_empty( ):
49        newest._next = newest                # initialize circularly
50      else:
51        newest._next = self._tail._next      # new node points to head
52        self._tail._next = newest            # old tail points to new node
53      self._tail = newest                    # new node becomes the tail
54      self._size += 1
55
56    def rotate(self):
57      """Rotate front element to the back of the queue."""
58      if self._size > 0:
59        self._tail = self._tail._next        # old head becomes new tail
```

Code Fragment 7.10: Implementation of a CircularQueue class, using a circularly linked list as storage (continued from Code Fragment 7.9).

7.3 Doubly Linked Lists

In a singly linked list, each node maintains a reference to the node that is immediately after it. We have demonstrated the usefulness of such a representation when managing a sequence of elements. However, there are limitations that stem from the asymmetry of a singly linked list. In the opening of Section 7.1, we emphasized that we can efficiently insert a node at either end of a singly linked list, and can delete a node at the head of a list, but we are unable to efficiently delete a node at the tail of the list. More generally, we cannot efficiently delete an arbitrary node from an interior position of the list if only given a reference to that node, because we cannot determine the node that immediately *precedes* the node to be deleted (yet, that node needs to have its next reference updated).

To provide greater symmetry, we define a linked list in which each node keeps an explicit reference to the node before it and a reference to the node after it. Such a structure is known as a ***doubly linked list***. These lists allow a greater variety of $O(1)$-time update operations, including insertions and deletions at arbitrary positions within the list. We continue to use the term "next" for the reference to the node that follows another, and we introduce the term "prev" for the reference to the node that precedes it.

Header and Trailer Sentinels

In order to avoid some special cases when operating near the boundaries of a doubly linked list, it helps to add special nodes at both ends of the list: a ***header*** node at the beginning of the list, and a ***trailer*** node at the end of the list. These "dummy" nodes are known as ***sentinels*** (or guards), and they do not store elements of the primary sequence. A doubly linked list with such sentinels is shown in Figure 7.10.

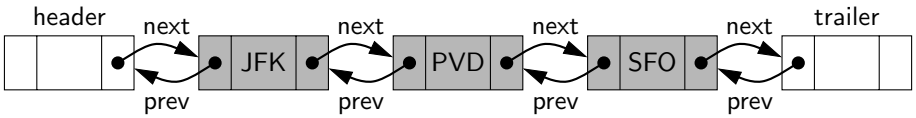


Figure 7.10: A doubly linked list representing the sequence { JFK, PVD, SFO }, using sentinels header and trailer to demarcate the ends of the list.

When using sentinel nodes, an empty list is initialized so that the next field of the header points to the trailer, and the prev field of the trailer points to the header; the remaining fields of the sentinels are irrelevant (presumably None, in Python). For a nonempty list, the header's next will refer to a node containing the first real element of a sequence, just as the trailer's prev references the node containing the last element of a sequence.

Advantage of Using Sentinels

Although we could implement a doubly linked list without sentinel nodes (as we did with our singly linked list in Section 7.1), the slight extra space devoted to the sentinels greatly simplifies the logic of our operations. Most notably, the header and trailer nodes never change—only the nodes between them change. Furthermore, we can treat all insertions in a unified manner, because a new node will always be placed between a pair of existing nodes. In similar fashion, every element that is to be deleted is guaranteed to be stored in a node that has neighbors on each side.

For contrast, look back at our LinkedQueue implementation from Section 7.1.2. Its enqueue method, given in Code Fragment 7.8, adds a new node to the end of the list. However, its implementation required a conditional to manage the special case of inserting into an empty list. In the general case, the new node was linked after the existing tail. But when adding to an empty list, there is no existing tail; instead it is necessary to reassign self._head to reference the new node. The use of a sentinel node in that implementation would eliminate the special case, as there would always be an existing node (possibly the header) before a new node.

Inserting and Deleting with a Doubly Linked List

Every insertion into our doubly linked list representation will take place between a pair of existing nodes, as diagrammed in Figure 7.11. For example, when a new element is inserted at the front of the sequence, we will simply add the new node *between* the header and the node that is currently after the header. (See Figure 7.12.)

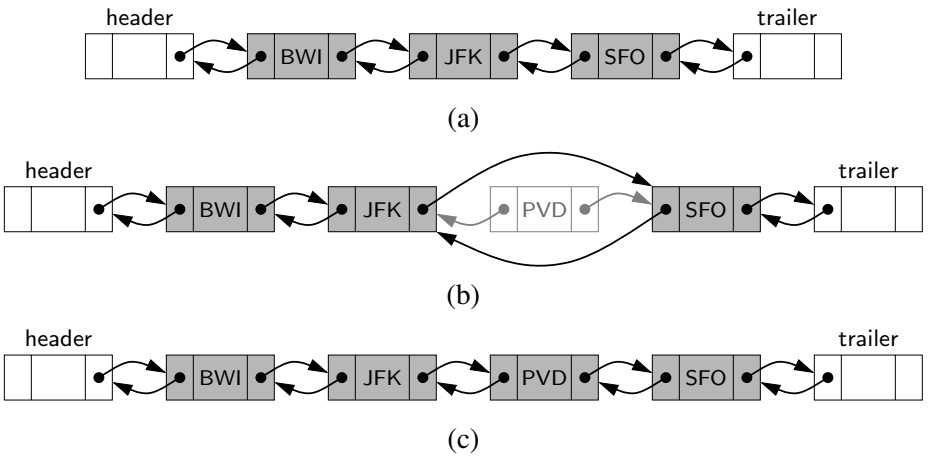


Figure 7.11: Adding an element to a doubly linked list with header and trailer sentinels: (a) before the operation; (b) after creating the new node; (c) after linking the neighbors to the new node.

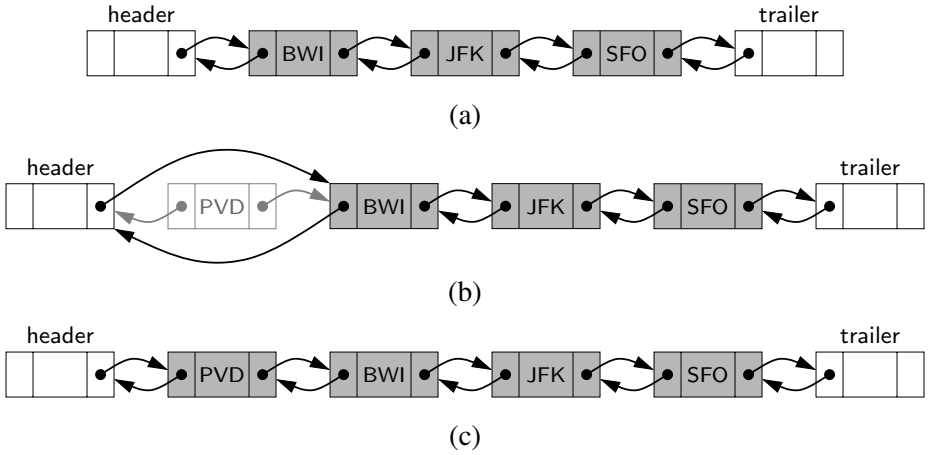


Figure 7.12: Adding an element to the front of a sequence represented by a doubly linked list with header and trailer sentinels: (a) before the operation; (b) after creating the new node; (c) after linking the neighbors to the new node.

The deletion of a node, portrayed in Figure 7.13, proceeds in the opposite fashion of an insertion. The two neighbors of the node to be deleted are linked directly to each other, thereby bypassing the original node. As a result, that node will no longer be considered part of the list and it can be reclaimed by the system. Because of our use of sentinels, the same implementation can be used when deleting the first or the last element of a sequence, because even such an element will be stored at a node that lies between two others.

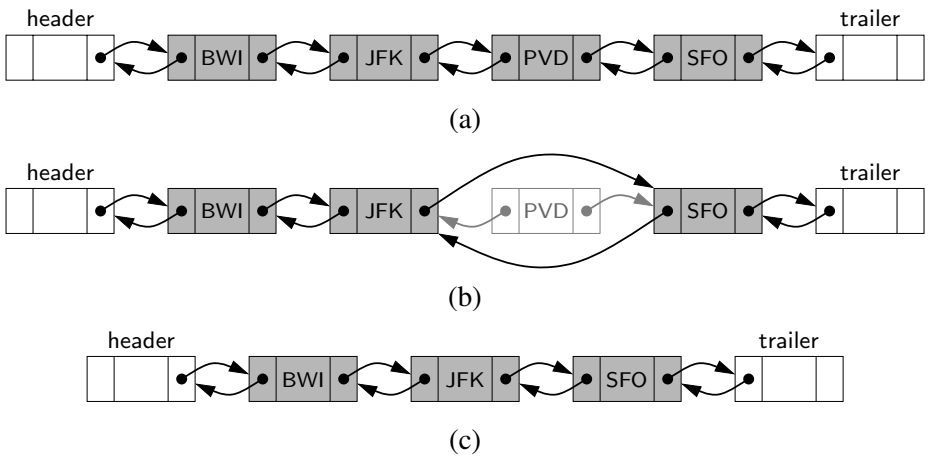


Figure 7.13: Removing the element PVD from a doubly linked list: (a) before the removal; (b) after linking out the old node; (c) after the removal (and garbage collection).

7.3.1 Basic Implementation of a Doubly Linked List

We begin by providing a preliminary implementation of a doubly linked list, in the form of a class named _DoublyLinkedBase. We intentionally name the class with a leading underscore because we do not intend for it to provide a coherent public interface for general use. We will see that linked lists can support general insertions and deletions in $O(1)$ worst-case time, but only if the location of an operation can be succinctly identified. With array-based sequences, an integer index was a convenient means for describing a position within a sequence. However, an index is not convenient for linked lists as there is no efficient way to find the jth element; it would seem to require a traversal of a portion of the list.

When working with a linked list, the most direct way to describe the location of an operation is by identifying a relevant node of the list. However, we prefer to encapsulate the inner workings of our data structure to avoid having users directly access nodes of a list. In the remainder of this chapter, we will develop two public classes that inherit from our _DoublyLinkedBase class to provide more coherent abstractions. Specifically, in Section 7.3.2, we provide a LinkedDeque class that implements the double-ended queue ADT introduced in Section 6.3; that class only supports operations at the ends of the queue, so there is no need for a user to identify an interior position within the list. In Section 7.4, we introduce a new PositionalList abstraction that provides a public interface that allows arbitrary insertions and deletions from a list.

Our low-level _DoublyLinkedBase class relies on the use of a nonpublic _Node class that is similar to that for a singly linked list, as given in Code Fragment 7.4, except that the doubly linked version includes a _prev attribute, in addition to the _next and _element attributes, as shown in Code Fragment 7.11.

```
class _Node:
  """Lightweight, nonpublic class for storing a doubly linked node."""
  __slots__ = '_element', '_prev', '_next'  # streamline memory

  def __init__(self, element, prev, next):          # initialize node's fields
    self._element = element                         # user's element
    self._prev = prev                               # previous node reference
    self._next = next                               # next node reference
```

Code Fragment 7.11: A Python _Node class for use in a doubly linked list.

The remainder of our _DoublyLinkedBase class is given in Code Fragment 7.12. The constructor instantiates the two sentinel nodes and links them directly to each other. We maintain a _size member and provide public support for __len__ and is_empty so that these behaviors can be directly inherited by the subclasses.

```
class _DoublyLinkedBase:
  """A base class providing a doubly linked list representation."""

  class _Node:
    """Lightweight, nonpublic class for storing a doubly linked node."""
    (omitted here; see previous code fragment)

  def __init__(self):
    """Create an empty list."""
    self._header = self._Node(None, None, None)
    self._trailer = self._Node(None, None, None)
    self._header._next = self._trailer                # trailer is after header
    self._trailer._prev = self._header                # header is before trailer
    self._size = 0                                    # number of elements

  def __len__(self):
    """Return the number of elements in the list."""
    return self._size

  def is_empty(self):
    """Return True if list is empty."""
    return self._size == 0

  def _insert_between(self, e, predecessor, successor):
    """Add element e between two existing nodes and return new node."""
    newest = self._Node(e, predecessor, successor)   # linked to neighbors
    predecessor._next = newest
    successor._prev = newest
    self._size += 1
    return newest

  def _delete_node(self, node):
    """Delete nonsentinel node from the list and return its element."""
    predecessor = node._prev
    successor = node._next
    predecessor._next = successor
    successor._prev = predecessor
    self._size -= 1
    element = node._element                           # record deleted element
    node._prev = node._next = node._element = None    # deprecate node
    return element                                    # return deleted element
```

Code Fragment 7.12: A base class for managing a doubly linked list.

The other two methods of our class are the nonpublic utilities, _insert_between and _delete_node. These provide generic support for insertions and deletions, respectively, but require one or more node references as parameters. The implementation of the _insert_between method is modeled upon the algorithm that was previously portrayed in Figure 7.11. It creates a new node, with that node's fields initialized to link to the specified neighboring nodes. Then the fields of the neighboring nodes are updated to include the newest node in the list. For later convenience, the method returns a reference to the newly created node.

The implementation of the _delete_node method is modeled upon the algorithm portrayed in Figure 7.13. The neighbors of the node to be deleted are linked directly to each other, thereby bypassing the deleted node from the list. As a formality, we intentionally reset the _prev, _next, and _element fields of the deleted node to None (after recording the element to be returned). Although the deleted node will be ignored by the rest of the list, setting its fields to None is advantageous as it may help Python's garbage collection, since unnecessary links to the other nodes and the stored element are eliminated. We will also rely on this configuration to recognize a node as "deprecated" when it is no longer part of the list.

7.3.2 Implementing a Deque with a Doubly Linked List

The double-ended queue (deque) ADT was introduced in Section 6.3. With an array-based implementation, we achieve all operations in *amortized* $O(1)$ time, due to the occasional need to resize the array. With an implementation based upon a doubly linked list, we can achieve all deque operation in *worst-case* $O(1)$ time.

We provide an implementation of a LinkedDeque class (Code Fragment 7.13) that inherits from the _DoublyLinkedBase class of the preceding section. We do not provide an explicit __init__ method for the LinkedDeque class, as the inherited version of that method suffices to initialize a new instance. We also rely on the inherited methods __len__ and is_empty in meeting the deque ADT.

With the use of sentinels, the key to our implementation is to remember that the header does not store the first element of the deque—it is the node just *after* the header that stores the first element (assuming the deque is nonempty). Similarly, the node just *before* the trailer stores the last element of the deque.

We use the inherited _insert_between method to insert at either end of the deque. To insert an element at the front of the deque, we place it immediately between the header and the node just after the header. An insertion at the end of deque is placed immediately before the trailer node. Note that these operations succeed, even when the deque is empty; in such a situation, the new node is placed between the two sentinels. When deleting an element from a nonempty deque, we rely upon the inherited _delete_node method, knowing that the designated node is assured to have neighbors on each side.

```
1  class LinkedDeque(_DoublyLinkedBase):          # note the use of inheritance
2    """Double-ended queue implementation based on a doubly linked list."""
3
4    def first(self):
5      """Return (but do not remove) the element at the front of the deque."""
6      if self.is_empty( ):
7        raise Empty("Deque is empty")
8      return self._header._next._element           # real item just after header
9
10   def last(self):
11     """Return (but do not remove) the element at the back of the deque."""
12     if self.is_empty( ):
13       raise Empty("Deque is empty")
14     return self._trailer._prev._element          # real item just before trailer
15
16   def insert_first(self, e):
17     """Add an element to the front of the deque."""
18     self._insert_between(e, self._header, self._header._next)   # after header
19
20   def insert_last(self, e):
21     """Add an element to the back of the deque."""
22     self._insert_between(e, self._trailer._prev, self._trailer)    # before trailer
23
24   def delete_first(self):
25     """Remove and return the element from the front of the deque.
26
27     Raise Empty exception if the deque is empty.
28     """
29     if self.is_empty( ):
30       raise Empty("Deque is empty")
31     return self._delete_node(self._header._next)       # use inherited method
32
33   def delete_last(self):
34     """Remove and return the element from the back of the deque.
35
36     Raise Empty exception if the deque is empty.
37     """
38     if self.is_empty( ):
39       raise Empty("Deque is empty")
40     return self._delete_node(self._trailer._prev)      # use inherited method
```

Code Fragment 7.13: Implementation of a LinkedDeque class that inherits from the _DoublyLinkedBase class.

7.4 The Positional List ADT

The abstract data types that we have considered thus far, namely stacks, queues, and double-ended queues, only allow update operations that occur at one end of a sequence or the other. We wish to have a more general abstraction. For example, although we motivated the FIFO semantics of a queue as a model for customers who are waiting to speak with a customer service representative, or fans who are waiting in line to buy tickets to a show, the queue ADT is too limiting. What if a waiting customer decides to hang up before reaching the front of the customer service queue? Or what if someone who is waiting in line to buy tickets allows a friend to "cut" into line at that position? We would like to design an abstract data type that provides a user a way to refer to elements anywhere in a sequence, and to perform arbitrary insertions and deletions.

When working with array-based sequences (such as a Python list), integer indices provide an excellent means for describing the location of an element, or the location at which an insertion or deletion should take place. However, numeric indices are not a good choice for describing positions within a linked list because we cannot efficiently access an entry knowing only its index; finding an element at a given index within a linked list requires traversing the list incrementally from its beginning or end, counting elements as we go.

Furthermore, indices are not a good abstraction for describing a local position in some applications, because the index of an entry changes over time due to insertions or deletions that happen earlier in the sequence. For example, it may not be convenient to describe the location of a person waiting in line by knowing precisely how far away that person is from the front of the line. We prefer an abstraction, as characterized in Figure 7.14, in which there is some other means for describing a position. We then wish to model situations such as when an identified person leaves the line before reaching the front, or in which a new person is added to a line immediately behind another identified person.

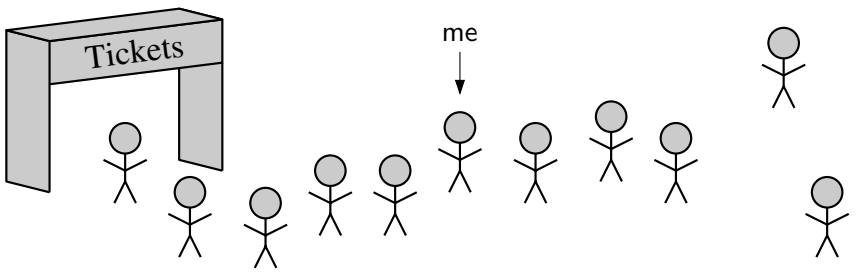


Figure 7.14: We wish to be able to identify the position of an element in a sequence without the use of an integer index.

As another example, a text document can be viewed as a long sequence of characters. A word processor uses the abstraction of a ***cursor*** to describe a position within the document without explicit use of an integer index, allowing operations such as "delete the character at the cursor" or "insert a new character just after the cursor." Furthermore, we may be able to refer to an inherent position within a document, such as the beginning of a particular section, without relying on a character index (or even a section number) that may change as the document evolves.

A Node Reference as a Position?

One of the great benefits of a linked list structure is that it is possible to perform $O(1)$-time insertions and deletions at arbitrary positions of the list, as long as we are given a reference to a relevant node of the list. It is therefore very tempting to develop an ADT in which a node reference serves as the mechanism for describing a position. In fact, our _DoublyLinkedBase class of Section 7.3.1 has methods _insert_between and _delete_node that accept node references as parameters.

However, such direct use of nodes would violate the object-oriented design principles of abstraction and encapsulation that were introduced in Chapter 2. There are several reasons to prefer that we encapsulate the nodes of a linked list, for both our sake and for the benefit of users of our abstraction.

- It will be simpler for users of our data structure if they are not bothered with unnecessary details of our implementation, such as low-level manipulation of nodes, or our reliance on the use of sentinel nodes. Notice that to use the _insert_between method of our _DoublyLinkedBase class to add a node at the beginning of a sequence, the header sentinel must be sent as a parameter.
- We can provide a more robust data structure if we do not permit users to directly access or manipulate the nodes. In that way, we ensure that users cannot invalidate the consistency of a list by mismanaging the linking of nodes. A more subtle problem arises if a user were allowed to call the _insert_between or _delete_node method of our _DoublyLinkedBase class, sending a node that does not belong to the given list as a parameter. (Go back and look at that code and see why it causes a problem!)
- By better encapsulating the internal details of our implementation, we have greater flexibility to redesign the data structure and improve its performance. In fact, with a well-designed abstraction, we can provide a notion of a non-numeric position, even if using an array-based sequence.

For these reasons, instead of relying directly on nodes, we introduce an independent ***position*** abstraction to denote the location of an element within a list, and then a complete ***positional list ADT*** that can encapsulate a doubly linked list (or even an array-based sequence; see Exercise P-7.46).

7.4.1 The Positional List Abstract Data Type

To provide for a general abstraction of a sequence of elements with the ability to identify the location of an element, we define a ***positional list ADT*** as well as a simpler ***position*** abstract data type to describe a location within a list. A position acts as a marker or token within the broader positional list. A position p is unaffected by changes elsewhere in a list; the only way in which a position becomes invalid is if an explicit command is issued to delete it.

A position instance is a simple object, supporting only the following method:

p.element( ): Return the element stored at position p.

In the context of the positional list ADT, positions serve as parameters to some methods and as return values from other methods. In describing the behaviors of a positional list, we being by presenting the accessor methods supported by a list L:

L.first( ): Return the position of the first element of L, or None if L is empty.

L.last( ): Return the position of the last element of L, or None if L is empty.

L.before(p): Return the position of L immediately before position p, or None if p is the first position.

L.after(p): Return the position of L immediately after position p, or None if p is the last position.

L.is_empty( ): Return True if list L does not contain any elements.

len(L): Return the number of elements in the list.

iter(L): Return a forward iterator for the *elements* of the list. See Section 1.8 for discussion of iterators in Python.

The positional list ADT also includes the following ***update*** methods:

L.add_first(e): Insert a new element e at the front of L, returning the position of the new element.

L.add_last(e): Insert a new element e at the back of L, returning the position of the new element.

L.add_before(p, e): Insert a new element e just before position p in L, returning the position of the new element.

L.add_after(p, e): Insert a new element e just after position p in L, returning the position of the new element.

L.replace(p, e): Replace the element at position p with element e, returning the element formerly at position p.

L.delete(p): Remove and return the element at position p in L, invalidating the position.

For those methods of the ADT that accept a position p as a parameter, an error occurs if p is not a valid position for list L.

Note well that the first( ) and last( ) methods of the positional list ADT return the associated *positions*, not the *elements*. (This is in contrast to the corresponding first and last methods of the deque ADT.) The first element of a positional list can be determined by subsequently invoking the element method on that position, as L.first( ).element( ). The advantage of receiving a position as a return value is that we can use that position to navigate the list. For example, the following code fragment prints all elements of a positional list named data.

```
cursor = data.first( )
while cursor is not None:
  print(cursor.element( ))        # print the element stored at the position
  cursor = data.after(cursor)     # advance to the next position (if any)
```

This code relies on the stated convention that the None object is returned when after is called upon the last position. That return value is clearly distinguishable from any legitimate position. The positional list ADT similarly indicates that the None value is returned when the before method is invoked at the front of the list, or when first or last methods are called upon an empty list. Therefore, the above code fragment works correctly even if the data list is empty.

Because the ADT includes support for Python's iter function, users may rely on the traditional for-loop syntax for such a forward traversal of a list named data.

```
for e in data:
  print(e)
```

More general navigational and update methods of the positional list ADT are shown in the following example.

Example 7.1: *The following table shows a series of operations on an initially empty positional list L. To identify position instances, we use variables such as p and q. For ease of exposition, when displaying the list contents, we use subscript notation to denote its positions.*

Operation	Return Value	L
L.add_last(8)	p	8_p
L.first( )	p	8_p
L.add_after(p, 5)	q	$8_p, 5_q$
L.before(q)	p	$8_p, 5_q$
L.add_before(q, 3)	r	$8_p, 3_r, 5_q$
r.element( )	3	$8_p, 3_r, 5_q$
L.after(p)	r	$8_p, 3_r, 5_q$
L.before(p)	None	$8_p, 3_r, 5_q$
L.add_first(9)	s	$9_s, 8_p, 3_r, 5_q$
L.delete(L.last( ))	5	$9_s, 8_p, 3_r$
L.replace(p, 7)	8	$9_s, 7_p, 3_r$

7.4.2 Doubly Linked List Implementation

In this section, we present a complete implementation of a PositionalList class using a doubly linked list that satisfies the following important proposition.

Proposition 7.2: *Each method of the positional list ADT runs in worst-case $O(1)$ time when implemented with a doubly linked list.*

We rely on the _DoublyLinkedBase class from Section 7.3.1 for our low-level representation; the primary responsibility of our new class is to provide a public interface in accordance with the positional list ADT. We begin our class definition in Code Fragment 7.14 with the definition of the public Position class, nested within our PositionalList class. Position instances will be used to represent the locations of elements within the list. Our various PositionalList methods may end up creating redundant Position instances that reference the same underlying node (for example, when first and last are the same). For that reason, our Position class defines the __eq__ and __ne__ special methods so that a test such as p == q evaluates to True when two positions refer to the same node.

Validating Positions

Each time a method of the PositionalList class accepts a position as a parameter, we want to verify that the position is valid, and if so, to determine the underlying node associated with the position. This functionality is implemented by a non-public method named _validate. Internally, a position maintains a reference to the associated node of the linked list, and also a reference to the list instance that contains the specified node. With the container reference, we can robustly detect when a caller sends a position instance that does not belong to the indicated list.

We are also able to detect a position instance that belongs to the list, but that refers to a node that is no longer part of that list. Recall that the _delete_node of the base class sets the previous and next references of a deleted node to None; we can recognize that condition to detect a deprecated node.

Access and Update Methods

The access methods of the PositionalList class are given in Code Fragment 7.15 and the update methods are given in Code Fragment 7.16. All of these methods trivially adapt the underlying doubly linked list implementation to support the public interface of the positional list ADT. Those methods rely on the _validate utility to "unwrap" any position that is sent. They also rely on a _make_position utility to "wrap" nodes as Position instances to return to the user, making sure never to return a position referencing a sentinel. For convenience, we have overridden the inherited _insert_between utility method so that ours returns a *position* associated with the newly created node (whereas the inherited version returns the node itself).

```
class PositionalList(_DoublyLinkedBase):
  """A sequential container of elements allowing positional access."""

  #------------------------- nested Position class -------------------------
  class Position:
    """An abstraction representing the location of a single element."""

    def __init__(self, container, node):
      """Constructor should not be invoked by user."""
      self._container = container
      self._node = node

    def element(self):
      """Return the element stored at this Position."""
      return self._node._element

    def __eq__(self, other):
      """Return True if other is a Position representing the same location."""
      return type(other) is type(self) and other._node is self._node

    def __ne__(self, other):
      """Return True if other does not represent the same location."""
      return not (self == other)          # opposite of __eq__

  #------------------------------ utility method ------------------------------
  def _validate(self, p):
    """Return position's node, or raise appropriate error if invalid."""
    if not isinstance(p, self.Position):
      raise TypeError('p must be proper Position type')
    if p._container is not self:
      raise ValueError('p does not belong to this container')
    if p._node._next is None:             # convention for deprecated nodes
      raise ValueError('p is no longer valid')
    return p._node
```

Code Fragment 7.14: A PositionalList class based on a doubly linked list. (Continues in Code Fragments 7.15 and 7.16.)

```
35    #------------------------------ utility method ------------------------------
36    def _make_position(self, node):
37      """Return Position instance for given node (or None if sentinel)."""
38      if node is self._header or node is self._trailer:
39        return None                                  # boundary violation
40      else:
41        return self.Position(self, node)             # legitimate position
42
43    #------------------------------ accessors ------------------------------
44    def first(self):
45      """Return the first Position in the list (or None if list is empty)."""
46      return self._make_position(self._header._next)
47
48    def last(self):
49      """Return the last Position in the list (or None if list is empty)."""
50      return self._make_position(self._trailer._prev)
51
52    def before(self, p):
53      """Return the Position just before Position p (or None if p is first)."""
54      node = self._validate(p)
55      return self._make_position(node._prev)
56
57    def after(self, p):
58      """Return the Position just after Position p (or None if p is last)."""
59      node = self._validate(p)
60      return self._make_position(node._next)
61
62    def __iter__(self):
63      """Generate a forward iteration of the elements of the list."""
64      cursor = self.first( )
65      while cursor is not None:
66        yield cursor.element( )
67        cursor = self.after(cursor)
```

Code Fragment 7.15: A PositionalList class based on a doubly linked list. (Continued from Code Fragment 7.14; continues in Code Fragment 7.16.)

```
68    #------------------------------ mutators ------------------------------
69    # override inherited version to return Position, rather than Node
70    def _insert_between(self, e, predecessor, successor):
71      """Add element between existing nodes and return new Position."""
72      node = super( )._insert_between(e, predecessor, successor)
73      return self._make_position(node)
74
75    def add_first(self, e):
76      """Insert element e at the front of the list and return new Position."""
77      return self._insert_between(e, self._header, self._header._next)
78
79    def add_last(self, e):
80      """Insert element e at the back of the list and return new Position."""
81      return self._insert_between(e, self._trailer._prev, self._trailer)
82
83    def add_before(self, p, e):
84      """Insert element e into list before Position p and return new Position."""
85      original = self._validate(p)
86      return self._insert_between(e, original._prev, original)
87
88    def add_after(self, p, e):
89      """Insert element e into list after Position p and return new Position."""
90      original = self._validate(p)
91      return self._insert_between(e, original, original._next)
92
93    def delete(self, p):
94      """Remove and return the element at Position p."""
95      original = self._validate(p)
96      return self._delete_node(original)      # inherited method returns element
97
98    def replace(self, p, e):
99      """Replace the element at Position p with e.
100
101     Return the element formerly at Position p.
102     """
103     original = self._validate(p)
104     old_value = original._element           # temporarily store old element
105     original._element = e                   # replace with new element
106     return old_value                        # return the old element value
```

Code Fragment 7.16: A PositionalList class based on a doubly linked list. (Continued from Code Fragments 7.14 and 7.15.)

7.5 Sorting a Positional List

In Section 5.5.2, we introduced the ***insertion-sort*** algorithm, in the context of an array-based sequence. In this section, we develop an implementation that operates on a PositionalList, relying on the same high-level algorithm in which each element is placed relative to a growing collection of previously sorted elements.

We maintain a variable named marker that represents the rightmost position of the currently sorted portion of a list. During each pass, we consider the position just past the marker as the pivot and consider where the pivot's element belongs relative to the sorted portion; we use another variable, named walk, to move leftward from the marker, as long as there remains a preceding element with value larger than the pivot's. A typical configuration of these variables is diagrammed in Figure 7.15. A Python implementation of this strategy is given in Code 7.17.

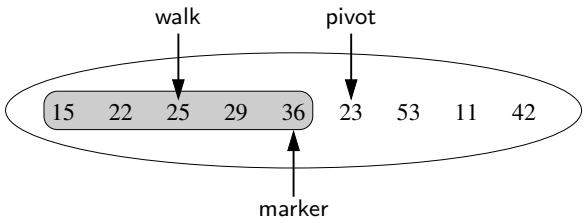


Figure 7.15: Overview of one step of our insertion-sort algorithm. The shaded elements, those up to and including marker, have already been sorted. In this step, the pivot's element should be relocated immediately before the walk position.

```
1  def insertion_sort(L):
2    """Sort PositionalList of comparable elements into nondecreasing order."""
3    if len(L) > 1:                          # otherwise, no need to sort it
4      marker = L.first( )
5      while marker != L.last( ):
6        pivot = L.after(marker)             # next item to place
7        value = pivot.element( )
8        if value > marker.element( ):       # pivot is already sorted
9          marker = pivot                    # pivot becomes new marker
10       else:                               # must relocate pivot
11         walk = marker                     # find leftmost item greater than value
12         while walk != L.first( ) and L.before(walk).element( ) > value:
13           walk = L.before(walk)
14         L.delete(pivot)
15         L.add_before(walk, value)         # reinsert value before walk
```

Code Fragment 7.17: Python code for performing insertion-sort on a positional list.

7.6 Case Study: Maintaining Access Frequencies

The positional list ADT is useful in a number of settings. For example, a program that simulates a game of cards could model each person's hand as a positional list (Exercise P-7.47). Since most people keep cards of the same suit together, inserting and removing cards from a person's hand could be implemented using the methods of the positional list ADT, with the positions being determined by a natural order of the suits. Likewise, a simple text editor embeds the notion of positional insertion and deletion, since such editors typically perform all updates relative to a ***cursor***, which represents the current position in the list of characters of text being edited.

In this section, we consider maintaining a collection of elements while keeping track of the number of times each element is accessed. Keeping such access counts allows us to know which elements are among the most popular. Examples of such scenarios include a Web browser that keeps track of a user's most accessed URLs, or a music collection that maintains a list of the most frequently played songs for a user. We model this with a new ***favorites list ADT*** that supports the len and is_empty methods as well as the following:

access(**e**): Access the element e, incrementing its access count, and adding it to the favorites list if it is not already present.

remove(**e**): Remove element e from the favorites list, if present.

top(**k**): Return an iteration of the k most accessed elements.

7.6.1 Using a Sorted List

Our first approach for managing a list of favorites is to store elements in a linked list, keeping them in nonincreasing order of access counts. We access or remove an element by searching the list from the most frequently accessed to the least frequently accessed. Reporting the top k most accessed elements is easy, as they are the first k entries of the list.

To maintain the invariant that elements are stored in nonincreasing order of access counts, we must consider how a single access operation may affect the order. The accessed element's count increases by one, and so it may become larger than one or more of its preceding neighbors in the list, thereby violating the invariant.

Fortunately, we can reestablish the sorted invariant using a technique similar to a single pass of the insertion-sort algorithm, introduced in the previous section. We can perform a backward traversal of the list, starting at the position of the element whose access count has increased, until we locate a valid position after which the element can be relocated.

Using the Composition Pattern

We wish to implement a favorites list by making use of a PositionalList for storage. If elements of the positional list were simply elements of the favorites list, we would be challenged to maintain access counts and to keep the proper count with the associated element as the contents of the list are reordered. We use a general object-oriented design pattern, the ***composition pattern***, in which we define a single object that is composed of two or more other objects. Specifically, we define a nonpublic nested class, _Item, that stores the element and its access count as a single instance. We then maintain our favorites list as a PositionalList of *item* instances, so that the access count for a user's element is embedded alongside it in our representation. (An _Item is never exposed to a user of a FavoritesList.)

```
1  class FavoritesList:
2    """List of elements ordered from most frequently accessed to least."""
3
4    #------------------------------ nested _Item class ------------------------------
5    class _Item:
6      __slots__ = '_value', '_count'              # streamline memory usage
7      def __init__(self, e):
8        self._value = e                           # the user's element
9        self._count = 0                           # access count initially zero
10
11   #------------------------------ nonpublic utilities ------------------------------
12   def _find_position(self, e):
13     """Search for element e and return its Position (or None if not found)."""
14     walk = self._data.first( )
15     while walk is not None and walk.element( )._value != e:
16       walk = self._data.after(walk)
17     return walk
18
19   def _move_up(self, p):
20     """Move item at Position p earlier in the list based on access count."""
21     if p != self._data.first( ):                  # consider moving...
22       cnt = p.element( )._count
23       walk = self._data.before(p)
24       if cnt > walk.element( )._count:           # must shift forward
25         while (walk != self._data.first( ) and
26                cnt > self._data.before(walk).element( )._count):
27           walk = self._data.before(walk)
28         self._data.add_before(walk, self._data.delete(p))     # delete/reinsert
```

Code Fragment 7.18: Class FavoritesList. (Continues in Code Fragment 7.19.)

```
29    #------------------------------ public methods ------------------------------
30    def __init__(self):
31      """Create an empty list of favorites."""
32      self._data = PositionalList( )                # will be list of _Item instances
33
34    def __len__(self):
35      """Return number of entries on favorites list."""
36      return len(self._data)
37
38    def is_empty(self):
39      """Return True if list is empty."""
40      return len(self._data) == 0
41
42    def access(self, e):
43      """Access element e, thereby increasing its access count."""
44      p = self._find_position(e)                    # try to locate existing element
45      if p is None:
46        p = self._data.add_last(self._Item(e))      # if new, place at end
47      p.element( )._count += 1                      # always increment count
48      self._move_up(p)                              # consider moving forward
49
50    def remove(self, e):
51      """Remove element e from the list of favorites."""
52      p = self._find_position(e)                    # try to locate existing element
53      if p is not None:
54        self._data.delete(p)                        # delete, if found
55
56    def top(self, k):
57      """Generate sequence of top k elements in terms of access count."""
58      if not 1 <= k <= len(self):
59        raise ValueError('Illegal value for k')
60      walk = self._data.first( )
61      for j in range(k):
62        item = walk.element( )                      # element of list is _Item
63        yield item._value                           # report user's element
64        walk = self._data.after(walk)
```

Code Fragment 7.19: Class FavoritesList. (Continued from Code Fragment 7.18.)

7.6.2 Using a List with the Move-to-Front Heuristic

The previous implementation of a favorites list performs the access(e) method in time proportional to the index of e in the favorites list. That is, if e is the kth most popular element in the favorites list, then accessing it takes $O(k)$ time. In many real-life access sequences (e.g., Web pages visited by a user), once an element is accessed it is more likely to be accessed again in the near future. Such scenarios are said to possess ***locality of reference***.

A ***heuristic***, or rule of thumb, that attempts to take advantage of the locality of reference that is present in an access sequence is the ***move-to-front heuristic***. To apply this heuristic, each time we access an element we move it all the way to the front of the list. Our hope, of course, is that this element will be accessed again in the near future. Consider, for example, a scenario in which we have n elements and the following series of n^2 accesses:

- element 1 is accessed n times
- element 2 is accessed n times
- $\cdots$
- element n is accessed n times.

If we store the elements sorted by their access counts, inserting each element the first time it is accessed, then

- each access to element 1 runs in $O(1)$ time
- each access to element 2 runs in $O(2)$ time
- $\cdots$
- each access to element n runs in $O(n)$ time.

Thus, the total time for performing the series of accesses is proportional to

$$n + 2n + 3n + \cdots + n \cdot n = n(1 + 2 + 3 + \cdots + n) = n \cdot \frac{n(n+1)}{2},$$

which is $O(n^3)$.

On the other hand, if we use the move-to-front heuristic, inserting each element the first time it is accessed, then

- each subsequent access to element 1 takes $O(1)$ time
- each subsequent access to element 2 takes $O(1)$ time
- $\cdots$
- each subsequent access to element n runs in $O(1)$ time.

So the running time for performing all the accesses in this case is $O(n^2)$. Thus, the move-to-front implementation has faster access times for this scenario. Still, the move-to-front approach is just a heuristic, for there are access sequences where using the move-to-front approach is slower than simply keeping the favorites list ordered by access counts.

The Trade-Offs with the Move-to-Front Heuristic

If we no longer maintain the elements of the favorites list ordered by their access counts, when we are asked to find the k most accessed elements, we need to search for them. We will implement the top(k) method as follows:

1. We copy all entries of our favorites list into another list, named temp.
2. We scan the temp list k times. In each scan, we find the entry with the largest access count, remove this entry from temp, and report it in the results.

This implementation of method top takes $O(kn)$ time. Thus, when k is a constant, method top runs in $O(n)$ time. This occurs, for example, when we want to get the "top ten" list. However, if k is proportional to n, then top runs in $O(n^2)$ time. This occurs, for example, when we want a "top 25%" list.

In Chapter 9 we will introduce a data structure that will allow us to implement top in $O(n + k \log n)$ time (see Exercise P-9.54), and more advanced techniques could be used to perform top in $O(n + k \log k)$ time.

We could easily achieve $O(n \log n)$ time if we use a standard sorting algorithm to reorder the temporary list before reporting the top k (see Chapter 12); this approach would be preferred to the original in the case that k is $\Omega(\log n)$. (Recall the big-Omega notation introduced in Section 3.3.1 to give an asymptotic lower bound on the running time of an algorithm.) There is a more specialized sorting algorithm (see Section 12.4.2) that can take advantage of the fact that access counts are integers in order to achieve $O(n)$ time for top, for any value of k.

Implementing the Move-to-Front Heuristic in Python

We give an implementation of a favorites list using the move-to-front heuristic in Code Fragment 7.20. The new FavoritesListMTF class inherits most of its functionality from the original FavoritesList as a base class.

By our original design, the access method of the original class relies on a non-public utility named _move_up to enact the potential shifting of an element forward in the list, after its access count had been incremented. Therefore, we implement the move-to-front heuristic by simply overriding the _move_up method so that each accessed element is moved directly to the front of the list (if not already there). This action is easily implemented by means of the positional list ADT.

The more complex portion of our FavoritesListMTF class is the new definition for the top method. We rely on the first of the approaches outlined above, inserting copies of the items into a temporary list and then repeatedly finding, reporting, and removing an element that has the largest access count of those remaining.

```
class FavoritesListMTF(FavoritesList):
  """List of elements ordered with move-to-front heuristic."""

  # we override _move_up to provide move-to-front semantics
  def _move_up(self, p):
    """Move accessed item at Position p to front of list."""
    if p != self._data.first( ):
      self._data.add_first(self._data.delete(p))        # delete/reinsert

  # we override top because list is no longer sorted
  def top(self, k):
    """Generate sequence of top k elements in terms of access count."""
    if not 1 <= k <= len(self):
      raise ValueError('Illegal value for k')

    # we begin by making a copy of the original list
    temp = PositionalList( )
    for item in self._data:                  # positional lists support iteration
      temp.add_last(item)

    # we repeatedly find, report, and remove element with largest count
    for j in range(k):
      # find and report next highest from temp
      highPos = temp.first( )
      walk = temp.after(highPos)
      while walk is not None:
        if walk.element( )._count > highPos.element( )._count:
          highPos = walk
        walk = temp.after(walk)
      # we have found the element with highest count
      yield highPos.element( )._value                # report element to user
      temp.delete(highPos)                           # remove from temp list
```

Code Fragment 7.20: Class FavoritesListMTF implementing the move-to-front heuristic. This class extends FavoritesList (Code Fragments 7.18 and 7.19) and overrides methods _move_up and top.

7.7 Link-Based vs. Array-Based Sequences

We close this chapter by reflecting on the relative pros and cons of array-based and link-based data structures that have been introduced thus far. The dichotomy between these approaches presents a common design decision when choosing an appropriate implementation of a data structure. There is not a one-size-fits-all solution, as each offers distinct advantages and disadvantages.

Advantages of Array-Based Sequences

- *Arrays provide $O(1)$-time access to an element based on an integer index.* The ability to access the kth element for any k in $O(1)$ time is a hallmark advantage of arrays (see Section 5.2). In contrast, locating the kth element in a linked list requires $O(k)$ time to traverse the list from the beginning, or possibly $O(n-k)$ time, if traversing backward from the end of a doubly linked list.
- *Operations with equivalent asymptotic bounds typically run a constant factor more efficiently with an array-based structure versus a linked structure.* As an example, consider the typical enqueue operation for a queue. Ignoring the issue of resizing an array, this operation for the ArrayQueue class (see Code Fragment 6.7) involves an arithmetic calculation of the new index, an increment of an integer, and storing a reference to the element in the array. In contrast, the process for a LinkedQueue (see Code Fragment 7.8) requires the instantiation of a node, appropriate linking of nodes, and an increment of an integer. While this operation completes in $O(1)$ time in either model, the actual number of CPU operations will be more in the linked version, especially given the instantiation of the new node.
- *Array-based representations typically use proportionally less memory than linked structures.* This advantage may seem counterintuitive, especially given that the length of a dynamic array may be longer than the number of elements that it stores. Both array-based lists and linked lists are referential structures, so the primary memory for storing the actual objects that are elements is the same for either structure. What differs is the auxiliary amounts of memory that are used by the two structures. For an array-based container of n elements, a typical worst case may be that a recently resized dynamic array has allocated memory for $2n$ object references. With linked lists, memory must be devoted not only to store a reference to each contained object, but also explicit references that link the nodes. So a singly linked list of length n already requires $2n$ references (an element reference and next reference for each node). With a doubly linked list, there are $3n$ references.

Advantages of Link-Based Sequences

- *Link-based structures provide worst-case time bounds for their operations.* This is in contrast to the amortized bounds associated with the expansion or contraction of a dynamic array (see Section 5.3).

  When many individual operations are part of a larger computation, and we only care about the total time of that computation, an amortized bound is as good as a worst-case bound precisely because it gives a guarantee on the sum of the time spent on the individual operations.

  However, if data structure operations are used in a real-time system that is designed to provide more immediate responses (e.g., an operating system, Web server, air traffic control system), a long delay caused by a single (amortized) operation may have an adverse effect.

- *Link-based structures support $O(1)$-time insertions and deletions at arbitrary positions.* The ability to perform a constant-time insertion or deletion with the PositionalList class, by using a Position to efficiently describe the location of the operation, is perhaps the most significant advantage of the linked list.

  This is in stark contrast to an array-based sequence. Ignoring the issue of resizing an array, inserting or deleting an element from the end of an array-based list can be done in constant time. However, more general insertions and deletions are expensive. For example, with Python's array-based list class, a call to insert or pop with index k uses $O(n-k+1)$ time because of the loop to shift all subsequent elements (see Section 5.4).

  As an example application, consider a text editor that maintains a document as a sequence of characters. Although users often add characters to the end of the document, it is also possible to use the cursor to insert or delete one or more characters at an arbitrary position within the document. If the character sequence were stored in an array-based sequence (such as a Python list), each such edit operation may require linearly many characters to be shifted, leading to $O(n)$ performance for each edit operation. With a linked-list representation, an arbitrary edit operation (insertion or deletion of a character at the cursor) can be performed in $O(1)$ worst-case time, assuming we are given a position that represents the location of the cursor.

7.8 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-7.1 Give an algorithm for finding the second-to-last node in a singly linked list in which the last node is indicated by a next reference of None.

R-7.2 Describe a good algorithm for concatenating two singly linked lists L and M, given only references to the first node of each list, into a single list L' that contains all the nodes of L followed by all the nodes of M.

R-7.3 Describe a recursive algorithm that counts the number of nodes in a singly linked list.

R-7.4 Describe in detail how to swap two nodes x and y (and not just their contents) in a singly linked list L given references only to x and y. Repeat this exercise for the case when L is a doubly linked list. Which algorithm takes more time?

R-7.5 Implement a function that counts the number of nodes in a circularly linked list.

R-7.6 Suppose that x and y are references to nodes of circularly linked lists, although not necessarily the same list. Describe a fast algorithm for telling if x and y belong to the same list.

R-7.7 Our CircularQueue class of Section 7.2.2 provides a rotate( ) method that has semantics equivalent to Q.enqueue(Q.dequeue( )), for a nonempty queue. Implement such a method for the LinkedQueue class of Section 7.1.2 without the creation of any new nodes.

R-7.8 Describe a nonrecursive method for finding, by link hopping, the middle node of a doubly linked list with header and trailer sentinels. In the case of an even number of nodes, report the node slightly left of center as the "middle." (Note: This method must only use link hopping; it cannot use a counter.) What is the running time of this method?

R-7.9 Give a fast algorithm for concatenating two doubly linked lists L and M, with header and trailer sentinel nodes, into a single list L'.

R-7.10 There seems to be some redundancy in the repertoire of the positional list ADT, as the operation L.add_first(e) could be enacted by the alternative L.add_before(L.first( ), e). Likewise, L.add_last(e) might be performed as L.add_after(L.last( ), e). Explain why the methods add_first and add_last are necessary.

R-7.11 Implement a function, with calling syntax max(L), that returns the maximum element from a PositionalList instance L containing comparable elements.

R-7.12 Redo the previously problem with max as a method of the PositionalList class, so that calling syntax L.max( ) is supported.

R-7.13 Update the PositionalList class to support an additional method find(e), which returns the position of the (first occurrence of) element e in the list (or None if not found).

R-7.14 Repeat the previous process using recursion. Your method should not contain any loops. How much space does your method use in addition to the space used for L?

R-7.15 Provide support for a __reversed__ method of the PositionalList class that is similar to the given __iter__, but that iterates the elements in *reversed* order.

R-7.16 Describe an implementation of the PositionalList methods add_last and add_before realized by using only methods in the set {is_empty, first, last, prev, next, add_after, and add_first}.

R-7.17 In the FavoritesListMTF class, we rely on public methods of the positional list ADT to move an element of a list at position p to become the first element of the list, while keeping the relative order of the remaining elements unchanged. Internally, that combination of operations causes one node to be removed and a new node to be inserted. Augment the PositionalList class to support a new method, move_to_front(p), that accomplishes this goal more directly, by relinking the existing node.

R-7.18 Given the set of element $\{a,b,c,d,e,f\}$ stored in a list, show the final state of the list, assuming we use the move-to-front heuristic and access the elements according to the following sequence: $(a,b,c,d,e,f,a,c,f,b,d,e)$.

R-7.19 Suppose that we have made kn total accesses to the elements in a list L of n elements, for some integer $k \geq 1$. What are the minimum and maximum number of elements that have been accessed fewer than k times?

R-7.20 Let L be a list of n items maintained according to the move-to-front heuristic. Describe a series of $O(n)$ accesses that will reverse L.

R-7.21 Suppose we have an n-element list L maintained according to the move-to-front heuristic. Describe a sequence of n^2 accesses that is guaranteed to take $\Omega(n^3)$ time to perform on L.

R-7.22 Implement a clear( ) method for the FavoritesList class that returns the list to empty.

R-7.23 Implement a reset_counts( ) method for the FavoritesList class that resets all elements' access counts to zero (while leaving the order of the list unchanged).

Creativity

C-7.24 Give a complete implementation of the stack ADT using a singly linked list that includes a header sentinel.

C-7.25 Give a complete implementation of the queue ADT using a singly linked list that includes a header sentinel.

C-7.26 Implement a method, concatenate(Q2) for the LinkedQueue class that takes all elements of LinkedQueue Q2 and appends them to the end of the original queue. The operation should run in $O(1)$ time and should result in Q2 being an empty queue.

C-7.27 Give a recursive implementation of a singly linked list class, such that an instance of a nonempty list stores its first element and a reference to a list of remaining elements.

C-7.28 Describe a fast recursive algorithm for reversing a singly linked list.

C-7.29 Describe in detail an algorithm for reversing a singly linked list L using only a constant amount of additional space and not using any recursion.

C-7.30 Exercise P-6.35 describes a LeakyStack abstraction. Implement that ADT using a ***singly*** linked list for storage.

C-7.31 Design a *forward list* ADT that abstracts the operations on a singly linked list, much as the positional list ADT abstracts the use of a doubly linked list. Implement a ForwardList class that supports such an ADT.

C-7.32 Design a circular positional list ADT that abstracts a circularly linked list in the same way that the positional list ADT abstracts a doubly linked list, with a notion of a designated "cursor" position within the list.

C-7.33 Modify the _DoublyLinkedBase class to include a reverse method that reverses the order of the list, yet without creating or destroying any nodes.

C-7.34 Modify the PositionalList class to support a method swap(p, q) that causes the underlying nodes referenced by positions p and q to be exchanged for each other. Relink the existing nodes; do not create any new nodes.

C-7.35 To implement the iter method of the PositionalList class, we relied on the convenience of Python's generator syntax and the yield statement. Give an alternative implementation of iter by designing a nested iterator class. (See Section 2.3.4 for discussion of iterators.)

C-7.36 Give a complete implementation of the positional list ADT using a doubly linked list that does not include any sentinel nodes.

C-7.37 Implement a function that accepts a PositionalList L of n integers sorted in nondecreasing order, and another value V, and determines in $O(n)$ time if there are two elements of L that sum precisely to V. The function should return a pair of positions of such elements, if found, or None otherwise.

C-7.38 There is a simple, but inefficient, algorithm, called ***bubble-sort***, for sorting a list L of n comparable elements. This algorithm scans the list $n-1$ times, where, in each scan, the algorithm compares the current element with the next one and swaps them if they are out of order. Implement a bubble_sort function that takes a positional list L as a parameter. What is the running time of this algorithm, assuming the positional list is implemented with a doubly linked list?

C-7.39 To better model a FIFO queue in which entries may be deleted before reaching the front, design a PositionalQueue class that supports the complete queue ADT, yet with enqueue returning a position instance and support for a new method, delete(p), that removes the element associated with position p from the queue. You may use the adapter design pattern (Section 6.1.2), using a PositionalList as your storage.

C-7.40 Describe an efficient method for maintaining a favorites list L, with move-to-front heuristic, such that elements that have not been accessed in the most recent n accesses are automatically purged from the list.

C-7.41 Exercise C-5.29 introduces the notion of a ***natural join*** of two databases. Describe and analyze an efficient algorithm for computing the natural join of a linked list A of n pairs and a linked list B of m pairs.

C-7.42 Write a Scoreboard class that maintains the top 10 scores for a game application using a singly linked list, rather than the array that was used in Section 5.5.1.

C-7.43 Describe a method for performing a ***card shuffle*** of a list of $2n$ elements, by converting it into two lists. A card shuffle is a permutation where a list L is cut into two lists, L_1 and L_2, where L_1 is the first half of L and L_2 is the second half of L, and then these two lists are merged into one by taking the first element in L_1, then the first element in L_2, followed by the second element in L_1, the second element in L_2, and so on.

Projects

P-7.44 Write a simple text editor that stores and displays a string of characters using the positional list ADT, together with a cursor object that highlights a position in this string. A simple interface is to print the string and then to use a second line of output to underline the position of the cursor. Your editor should support the following operations:

- left: Move cursor left one character (do nothing if at beginning).
- right: Move cursor right one character (do nothing if at end).
- insert c: Insert the character c just after the cursor.
- delete: Delete the character just after the cursor (do nothing at end).

P-7.45 An array A is ***sparse*** if most of its entries are empty (i.e., None). A list L can be used to implement such an array efficiently. In particular, for each nonempty cell $A[i]$, we can store an entry (i,e) in L, where e is the element stored at $A[i]$. This approach allows us to represent A using $O(m)$ storage, where m is the number of nonempty entries in A. Provide such a SparseArray class that minimally supports methods __getitem__(j) and __setitem__(j, e) to provide standard indexing operations. Analyze the efficiency of these methods.

P-7.46 Although we have used a doubly linked list to implement the positional list ADT, it is possible to support the ADT with an array-based implementation. The key is to use the composition pattern and store a sequence of position items, where each item stores an element as well as that element's current index in the array. Whenever an element's place in the array is changed, the recorded index in the position must be updated to match. Given a complete class providing such an array-based implementation of the positional list ADT. What is the efficiency of the various operations?

P-7.47 Implement a CardHand class that supports a person arranging a group of cards in his or her hand. The simulator should represent the sequence of cards using a single positional list ADT so that cards of the same suit are kept together. Implement this strategy by means of four "fingers" into the hand, one for each of the suits of hearts, clubs, spades, and diamonds, so that adding a new card to the person's hand or playing a correct card from the hand can be done in constant time. The class should support the following methods:

- add_card(r, s): Add a new card with rank r and suit s to the hand.
- play(s): Remove and return a card of suit s from the player's hand; if there is no card of suit s, then remove and return an arbitrary card from the hand.
- __iter__(): Iterate through all cards currently in the hand.
- all_of_suit(s): Iterate through all cards of suit s that are currently in the hand.

Chapter Notes

A view of data structures as collections (and other principles of object-oriented design) can be found in object-oriented design books by Booch [17], Budd [20], Goldberg and Robson [42], and Liskov and Guttag [71]. Our positional list ADT is derived from the "position" abstraction introduced by Aho, Hopcroft, and Ullman [6], and the list ADT of Wood [104]. Implementations of linked lists are discussed by Knuth [64].